

ManyHands — Propuesta de Trabajo Final de Grado

Ingeniería en Sistemas de Información · Universidad Nacional del Sur

Descomposición recursiva de tareas de software y ejecución de subagentes paralelos en worktrees aislados, con ejecución consciente de conflictos. Diseño técnico, implementación de un MVP funcional y evaluación empírica del efecto de la granularidad de descomposición sobre la calidad del resultado.

1. Problema y motivación

Los agentes de código basados en modelos de lenguaje (Claude Code, Codex, Aider) ya resuelven tareas reales de ingeniería de software, pero su rendimiento se degrada cuando la tarea es grande: el contexto se satura, el objetivo se difumina y el agente comete más errores. Una práctica creciente en la industria para mitigar esto es ejecutar varios agentes en paralelo, cada uno en un *git worktree* aislado —un directorio de trabajo con su propia rama, partiendo del mismo repositorio.

Esta práctica resuelve el aislamiento físico de la ejecución, pero deja dos problemas abiertos:

1. **Cómo descomponer** una tarea grande en sub-tareas del tamaño adecuado para que cada agente trabaje con contexto mínimo y objetivo acotado. Hoy esa partición la hace el desarrollador a mano, o directamente no se hace.
2. **Qué ocurre al integrar.** Los worktrees evitan colisiones de directorio *durante* la ejecución, pero difieren el riesgo al momento del *merge*: dos agentes pueden producir cambios incompatibles que recién se manifiestan al combinar las ramas. Ese riesgo diferido es el costo oculto del paralelismo no coordinado.

Idea central del trabajo. Si una tarea grande se descompone recursivamente hasta sub-tareas mínimas ("hojas") con alcance claro, y esas hojas se ejecutan en paralelo de forma consciente de qué archivos y símbolos toca cada una, entonces cada agente trabaja en mejores condiciones y los conflictos de integración pueden anticiparse y manejarse en lugar de sufrirse al final. El núcleo del trabajo es diseñar ese mecanismo con solidez técnica y medir si efectivamente mejora el resultado, y a qué nivel de granularidad.

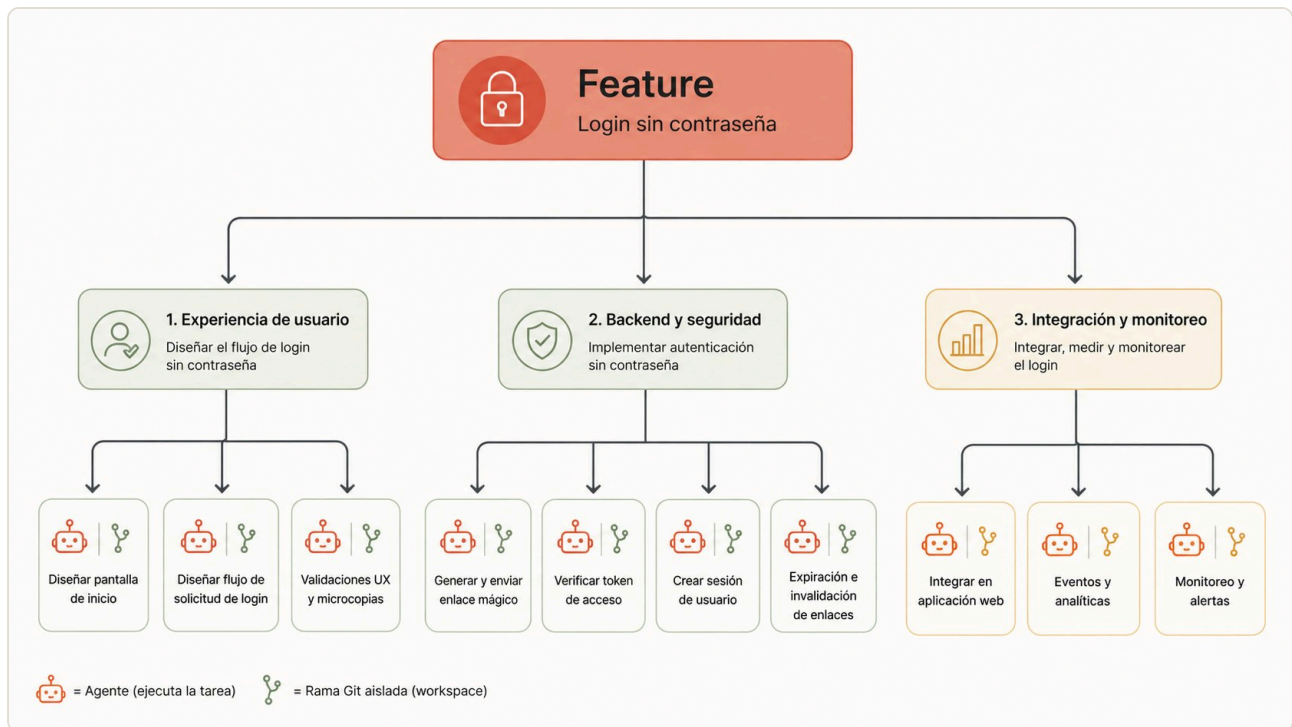


Figura 1. Concepto central: una feature grande se subdivide recursivamente en sub-tareas y finalmente en tareas hoja, cada una atendida por un agente en su propia rama de Git aislada.

Usuario y contexto de uso. Un desarrollador que coordina trabajo de agentes sobre un repositorio real, en un flujo con supervisión humana: aprueba el plan de descomposición, supervisa la ejecución y confirma la integración final. ManyHands potencia al ingeniero, no lo reemplaza.

2. Objetivo y pregunta de investigación

Objetivo general. Diseñar e implementar un sistema (ManyHands) que descomponga recursivamente una tarea de software en sub-tareas atómicas, las ejecute mediante subagentes en worktrees aislados con ejecución consciente de conflictos, e integre los resultados de abajo hacia arriba; y evaluar empíricamente el efecto de la granularidad de descomposición sobre la calidad del resultado.

Pregunta de investigación central.

¿Descomponer una tarea de software en sub-tareas atómicas y delegarlas a agentes paralelos produce un resultado de mejor calidad que un único agente con la tarea completa, y existe un nivel de granularidad que optimiza ese resultado?

Preguntas secundarias que la acompañan, a abordar según el tiempo disponible:

- ¿La ejecución consciente de conflictos —que evita correr en paralelo hojas que se solapan— reduce los fallos de integración frente al paralelismo no coordinado?
- ¿Cuál es la profundidad de descomposición práctica antes de que la sobrecarga de coordinación supere el beneficio?

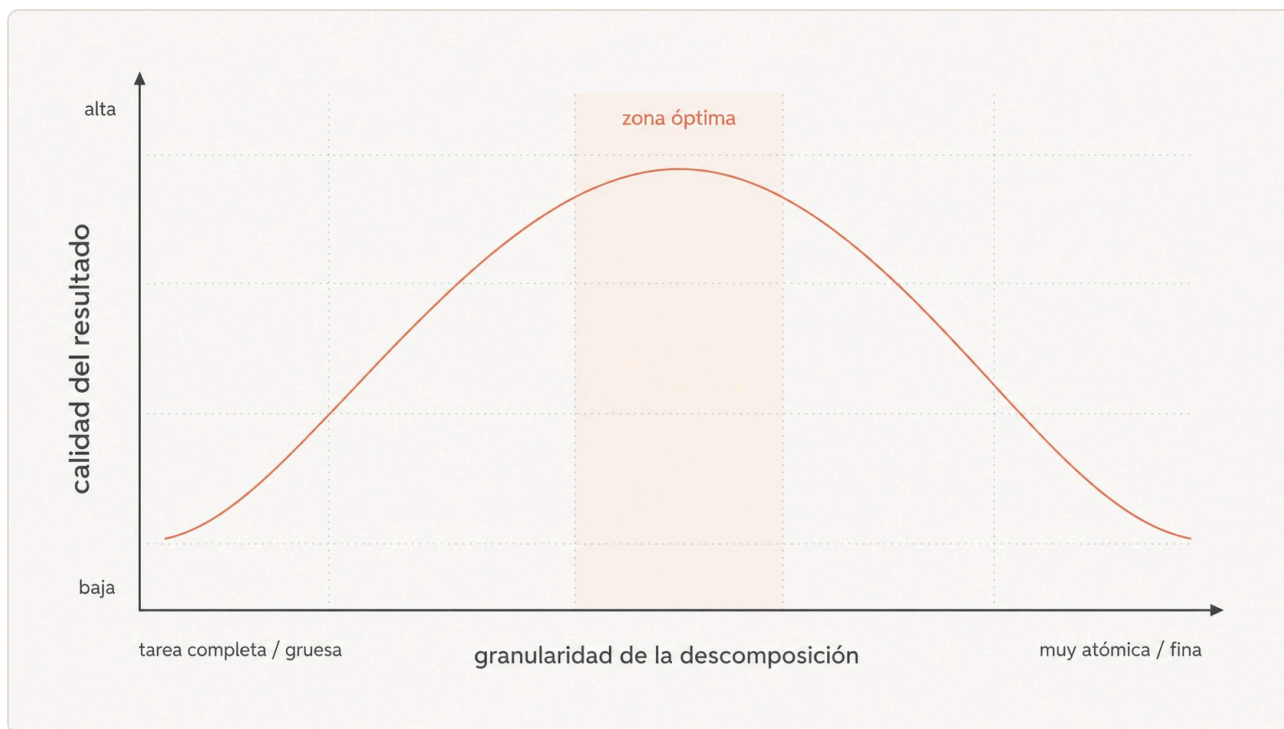


Figura 2. Relación esperada entre la granularidad de la descomposición y la calidad del resultado, con una zona intermedia óptima. Caracterizar esta curva es el objeto central de la evaluación.

Objetivos específicos.

1. Diseñar criterios operacionales de **atomicidad** y un algoritmo de descomposición recursiva (tarea → sub-tareas → hojas).
2. Diseñar el modelo de **contratos de tarea hoja** (objetivo, rutas permitidas/prohibidas, contexto, criterios de aceptación).
3. Implementar el **grafo de dependencias** (árbol de descomposición más dependencias transversales, es decir un DAG) con estados y readiness.
4. Implementar la **capa de ejecución en worktrees** con un agente real (Claude Code CLI) y validación local (compilación, lint, tests).
5. Diseñar e implementar la **ejecución consciente de conflictos**: análisis de solapamiento de archivos y símbolos para decidir qué hojas correr juntas y para detectar conflictos al integrar.
6. Implementar el **motor de integración de abajo hacia arriba** y la generación de un *diff* / borrador de *pull request* final.
7. Construir un **MVP funcional** con interfaz de usuario (modos Plan / Descomponer / Ejecutar / Integrar).
8. Diseñar y ejecutar la **evaluación empírica** de la pregunta central y analizar los resultados.

3. Alcance y límites

Dentro de alcance.

- Lenguaje objetivo: **TypeScript**. Su sistema de tipos es un activo técnico central del trabajo (ver sección 5).

- Un backend de agente real (Claude Code CLI) invocado como subprocesso.
- Repositorios de tamaño acotado con suite de tests existente, que provee un criterio objetivo de validación.
- Ejecución consciente de conflictos a nivel de archivos y símbolos.

Fuera de alcance.

- Entrenar o ajustar modelos: se utilizan a través de su API.
- Resolución *automática* de conflictos complejos: el sistema los detecta y propone estrategias o los delega al desarrollador; no se promete resolverlos sin intervención.
- Soporte multi-lenguaje, multiusuario o despliegue productivo.
- Garantías formales de completitud u optimalidad del algoritmo.

Profundidad y granularidad. La descomposición se limita a una profundidad práctica (aproximadamente tres a seis niveles), y cada hoja toca pocos archivos. La profundidad óptima es justamente objeto del experimento, no un parámetro fijado de antemano.

4. Posicionamiento respecto al trabajo existente

La idea de descomponer recursivamente una tarea hasta unidades atómicas tiene antecedentes en la literatura. *Guided Code Generation with LLMs* (arXiv:2501.06625) descompone recursivamente hasta hojas indivisibles y reporta una mejora del 23,79 % en Pass@1 sobre HumanEval; sin embargo, evalúa funciones aisladas en una sola máquina. ManyHands lleva esa idea a un escenario distinto:

repositorios reales, con worktrees paralelos e integración progresiva. Otros antecedentes relevantes son TDAG (arXiv:2402.10178), que descompone dinámicamente y genera un subagente por sub-tarea, y los frameworks multi-agente MetaGPT, ChatDev y AgentOrchestra (arXiv:2506.12508). El trabajo *Compositional Hardness of Code in LLMs* (arXiv:2409.18028) ofrece respaldo teórico a por qué descomponer ayuda: los modelos fallan en la composición de soluciones, y la subdivisión en sub-tareas mitiga esa dificultad.

En la industria existen orquestadores recientes —Conductor, Vibe Kanban, Crystal y Symphony (OpenAI, 2026)— que ejecutan agentes en paralelo sobre worktrees. La diferencia de ManyHands es doble: estos sistemas toman tareas escritas por humanos y las tratan como independientes, mientras que ManyHands **descompone automáticamente** la tarea y **modela las dependencias y los conflictos** entre sub-tareas; y delegan la resolución de conflictos y el merge enteramente al desarrollador, mientras que ManyHands los anticipa.

El aporte de este trabajo, a escala de un trabajo final de grado, es combinar estas piezas en un sistema coherente y, sobre todo, **medir el efecto de la granularidad de descomposición sobre la calidad**, un aspecto que los trabajos anteriores no estudian en este contexto. La metodología de evaluación se inspira en SWE-bench (Princeton), el estándar para evaluar agentes sobre issues reales de GitHub usando la suite de tests como criterio objetivo.

5. Diseño técnico

El peso del trabajo está en diseñar correctamente los componentes siguientes; cada uno implica decisiones de diseño no triviales que se justifican en el informe final.

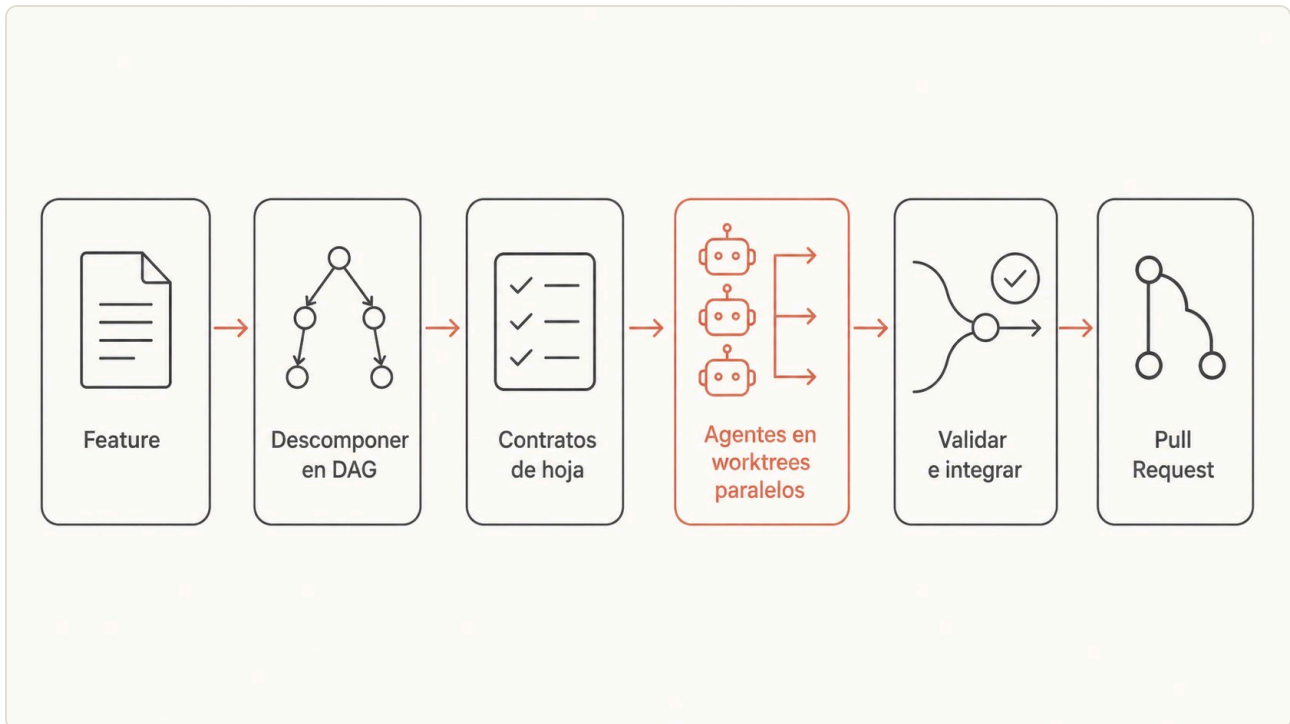


Figura 3. Flujo de extremo a extremo del sistema: de la feature al pull request, pasando por la descomposición en DAG, los contratos de hoja, la ejecución de agentes en worktrees paralelos y la integración validada.

5.1 Descomponedor recursivo y criterios de atomicidad

Un agente planificador que, dada la feature, produce un árbol de sub-tareas aplicando criterios de atomicidad hasta llegar a hojas. Una hoja debe cumplir: objetivo único explicable en una a tres frases; alcance de pocos archivos; límites negativos explícitos (qué no debe tocar); criterios de aceptación verificables; ausencia de decisiones arquitectónicas; contexto mínimo local; y salida integrable. La descomposición se realiza en dos fases —esqueleto general y luego refinamiento por nodo— y admite re-subdivisión bajo demanda del usuario. El riesgo de descomposiciones inconsistentes del modelo se mitiga con validación de esquema y con la posibilidad de edición humana del grafo.

5.2 Contratos de tarea hoja

Cada hoja se convierte en un encargo preciso para el agente: objetivo, rutas permitidas y prohibidas, contexto requerido (tipos y firmas relevantes), criterios de aceptación y salidas esperadas. El contrato limita el daño fuera de alcance y es la base del análisis de conflictos.

5.3 Grafo de dependencias

Combina la jerarquía de descomposición con dependencias transversales —por ejemplo, una hoja que produce un tipo de dato es prerequisite de otra en una rama distinta—. Por eso la estructura es un grafo dirigido acíclico (DAG) y no un árbol puro. El grafo computa qué hojas están listas o bloqueadas y agrega el estado en los nodos compuestos.

5.4 Ejecución consciente de conflictos

Antes de lanzar un lote de hojas en paralelo, ManyHands analiza qué archivos y símbolos declara y toca cada hoja —a partir de sus contratos y del análisis estático del repositorio— y evita ejecutar juntas hojas que se solapan. Al integrar, detecta conflictos textuales (vía git), estructurales (mismo símbolo) y de tipos o contratos. El activo técnico distintivo de este trabajo es utilizar la **API del compilador de TypeScript** no sólo como validador posterior, sino para verificar la compatibilidad de tipos y firmas entre ramas hermanas. Esto eleva la profundidad técnica del componente al nivel de las herramientas actuales.

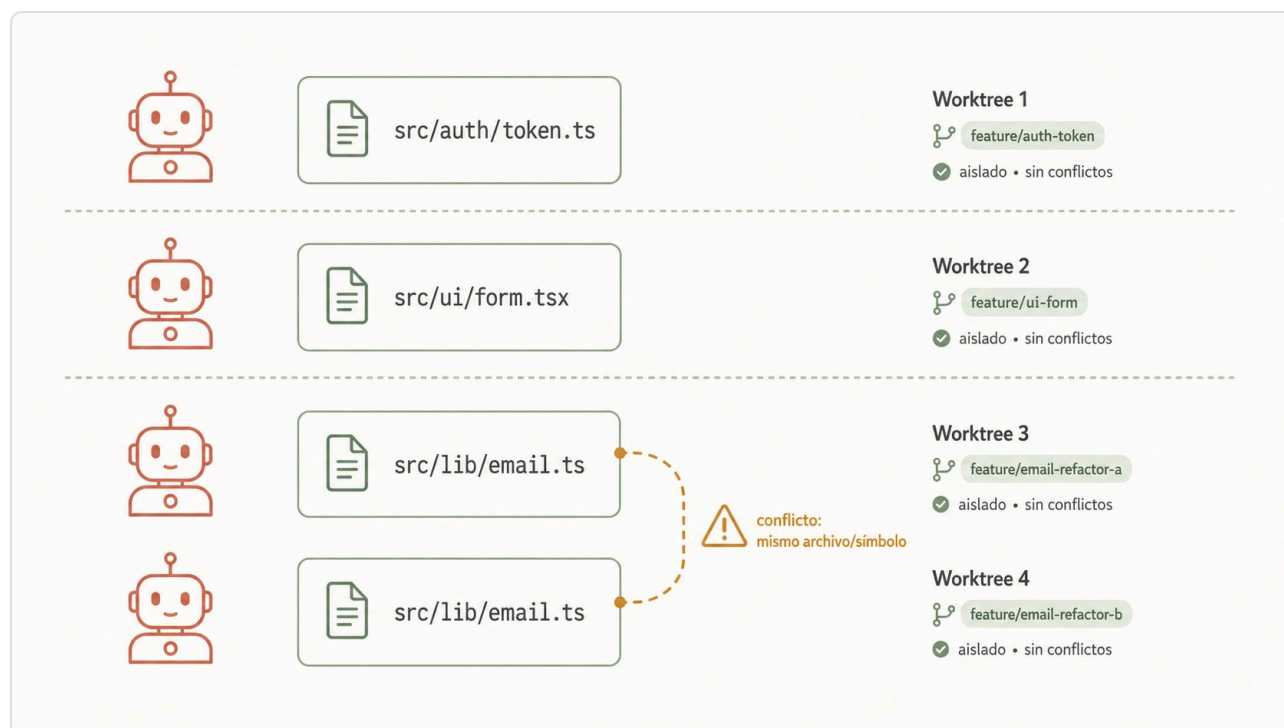


Figura 4. Ejecución paralela en worktrees aislados. El análisis de solapamiento detecta que dos agentes tocan el mismo archivo o símbolo y por lo tanto no deben ejecutarse en simultáneo; los demás avanzan en paralelo sin riesgo.

5.5 Capa de ejecución en worktrees

Por cada hoja lista, ManyHands crea un worktree y una rama dedicada (`git worktree add -b mh/<id>`), invoca al agente con el contrato dentro de ese directorio, recoge el `diff` y los logs, y corre la validación local (compilación de TypeScript, lint y tests unitarios). Si pasa, hace commit; si falla, la marca como fallida para reintento o re-subdivisión. Cada worktree aísla también el entorno de ejecución (puertos, variables de entorno, base de datos de prueba).

5.6 Integración de abajo hacia arriba

Cuando todas las hojas de un nodo compuesto pasan sus validaciones, se integran en una rama del subárbol, se resuelven los conflictos locales y se ejecutan los tests del módulo; sólo entonces el nodo padre se marca como completado. El proceso se repite recursivamente hasta la raíz, produciendo un `diff` final y un borrador de `pull request`.

5.7 Interfaz del MVP

Cuatro modos: **Plan** (describir la feature), **Descomponer** (árbol de tareas, inspector de nodo y contrato, re-subdivisión), **Ejecutar** (estados en vivo, lote actual, logs del agente) e **Integrar** (orden de

merge, conflictos, *diff* y *pull request*). La estética es técnica y centrada en el grafo de tareas. La Figura 5 muestra la vista principal de orquestación.

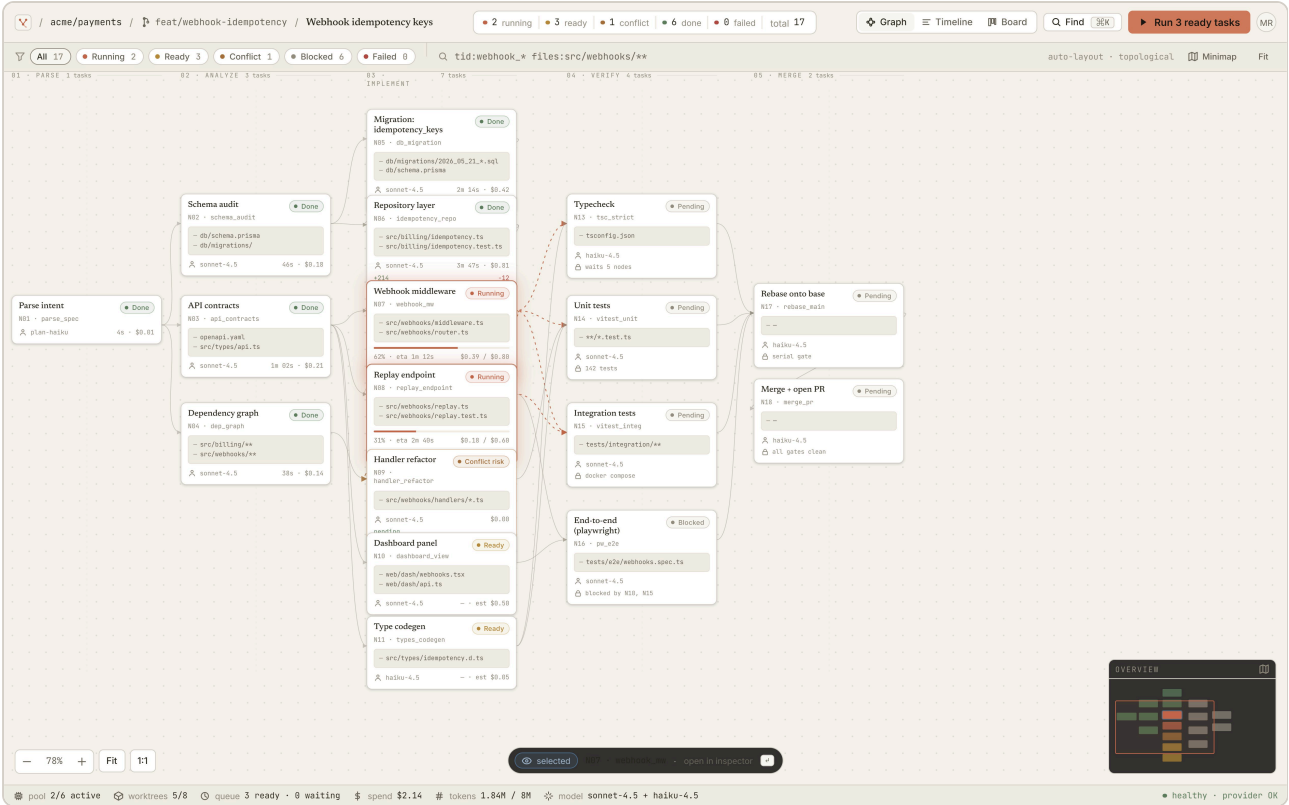


Figura 5. Vista principal del MVP: el grafo de tareas por fases (Parse · Analyze · Implement · Verify · Merge), con agentes en ejecución, dependencias, estados por nodo y advertencias de conflicto, junto con la barra de estado del sistema (agentes activos, worktrees, costo).

Modelo de datos. Las entidades principales son **Plan**, **Task** (con padre, intención, alcance, restricciones, criterio de aceptación, estado y worktree), **Edge** (dependencia entre tareas, con tipo), **Conflict** (par de tareas, nivel, archivos y símbolos compartidos), **Trace** (eventos de ejecución) e **IntegrationStep** (paso del merge).

6. Metodología de evaluación

Para responder la pregunta central se comparan distintas configuraciones sobre las mismas features:

- **C0 — Agente único:** la feature completa entregada a un solo agente (configuración de referencia).
- **C1 — Descomposición gruesa:** pocas sub-tareas grandes.
- **C2 — Descomposición media.**
- **C3 — Descomposición fina (atómica).**

Las políticas de ejecución —secuencial frente a paralela consciente de conflictos— se tratan como variable secundaria, a medir si el tiempo lo permite.

Tareas de prueba. Tres features de complejidad moderada en proyectos TypeScript **con suite de tests existente**, ya que el harness de tests provee un criterio de validación objetivo. Se eligen al inicio del trabajo (semana 1) verificando que sus tests sean buen indicador de correctitud. Casos candidatos:

autenticación sin contraseña con *magic links*; filtrado y ordenamiento en una vista de usuarios (backend más interfaz); y descuentos con persistencia en un carrito de compras.

Métricas.

- *Principal y objetiva*: **proporción de tests que pasan / éxito de integración** del resultado final.
- *Secundarias*: número de conflictos reales al integrar; re-trabajo (hojas re-ejecutadas); cobertura de tests; tiempo total y costo en tokens. Estas dos últimas se reportan con cautela, ya que dependen de la infraestructura y de los límites de tasa de la API.

Reproducibilidad y control de sesgos. Al menos tres repeticiones por configuración con semillas o prompts variados; versiones de modelo fijas; repositorios en *commits* fijos; entornos de ejecución aislados; y registro versionado de configuraciones y logs. Se reporta la **varianza** entre repeticiones, no una única corrida. Dada la variabilidad inherente de los modelos de lenguaje y el número acotado de tareas, los resultados se presentan e interpretan como un estudio de carácter exploratorio: se reportan tamaños de efecto y se consideran válidas las conclusiones matizadas (por ejemplo, "la técnica mejora el resultado en ciertos casos y no en otros"), que son en sí mismas un aporte.

Análisis. Curva de calidad en función de la granularidad, con bandas de varianza, por tarea y agregada; y gráficos comparativos de conflictos y tiempo por configuración.

7. Plan de trabajo (12 semanas)

Semana	Hito
1	Estado del arte; cierre de criterios de atomicidad y arquitectura. Selección de las features de prueba. Lazo mínimo de extremo a extremo: una feature, un agente, un worktree, tests.
2	Descomponedor (esqueleto) y esquema del grafo de tareas.
3	Contratos de tarea hoja; re-subdivisión bajo demanda.
4	Interfaz Plan y Descomponer (árbol e inspector).
5	Ejecución secuencial en worktrees con agente real y validación (compilación / tests).
6	Ejecución paralela y análisis de solapamiento de archivos y símbolos.
7	Detección de conflictos (git, compilador de tipos); manejo de fallos.
8	Integración de abajo hacia arriba; primera feature completa de extremo a extremo.
9	Interfaz Ejecutar (estados en vivo, logs).
10	Interfaz Integrar (<i>diff</i> y <i>pull request</i>); pulido del MVP.
11	Preparación experimental: configuraciones, métricas y corridas piloto.
12	Ejecución de experimentos, análisis de resultados, redacción del informe y preparación de la defensa.

La decisión metodológica temprana más importante es construir primero el lazo *descomponer* → *ejecutar* → *validar* sobre una sola feature real, antes de invertir en la interfaz completa o en el análisis de conflictos.

8. Riesgos y mitigaciones

- **Variabilidad de los modelos de lenguaje**, que dificulta obtener conclusiones con un número acotado de tareas. Mitigación: múltiples repeticiones, encuadre exploratorio y reporte de tamaños de efecto.
- **Calidad del descomponedor**, que es el componente más dependiente del modelo. Mitigación: validación de esquema y edición humana del grafo de tareas.
- **Ambición excesiva para tres meses**. Mitigación: MVP funcional pero acotado —un lenguaje, un agente, sin multiusuario—; el análisis de conflictos de tipos se profundiza si el tiempo lo permite, sin ser requisito de aprobación.
- **Tiempo y costo confundidos por la infraestructura**. Mitigación: la métrica principal es la proporción de tests que pasan, no el tiempo.
- **Percepción de la herramienta como un simple envoltorio de un modelo**. Mitigación: el peso del trabajo recae en el diseño técnico —descomponedor, grafo de dependencias y ejecución consciente de conflictos apoyada en el compilador de tipos— y en la medición empírica, no en la interfaz.

Se consideran simplificaciones aceptables trabajar con un solo lenguaje, tratar el análisis de conflictos de tipos como una heurística no probada formalmente, usar un dataset acotado y un único backend de agente. No se consideran aceptables la ausencia de evaluación empírica ni extraer conclusiones de una única corrida.

9. Stack técnico

El sistema se implementa íntegramente en **TypeScript**, unificando frontend y backend. La razón principal es que el activo técnico distintivo del trabajo —la verificación de compatibilidad de tipos entre ramas mediante la API del compilador de TypeScript— se integra de forma natural y en proceso en un entorno Node/TypeScript, evitando la fricción de cruzar lenguajes; además, unificar el stack reduce el costo de contexto para un desarrollo individual en tres meses.

- **Frontend**: Next.js con React y React Flow para el canvas del grafo de tareas, con Tailwind para los estilos.
- **Backend / orquestador**: Node con TypeScript.
- **Análisis estático**: API del compilador de TypeScript (`tsserver` / TypeScript Compiler API).
- **Agente de ejecución**: Claude Code CLI invocado como subprocesso; modelos a través de su API.
- **Persistencia**: SQLite, por su simplicidad, portabilidad y reproducibilidad.
- **Aislamiento**: *git worktree* nativo.
- **Comunicación en tiempo real**: WebSockets para el estado de las tareas.

10. Qué se demuestra en la defensa

1. Una **demostración en vivo**: una feature real que se descompone en un árbol de sub-tareas, se ejecuta mediante agentes en paralelo, se integra y produce un *pull request*.
2. El **diseño técnico** del descomponedor, el grafo de dependencias y la ejecución consciente de conflictos, con la verificación de tipos entre ramas como pieza distintiva.
3. La **relación entre granularidad y calidad** medida experimentalmente, con su análisis y sus límites.
4. Una discusión crítica de qué funcionó, qué no, y el trabajo futuro.

Apéndice A — Visión de producto (fuera del alcance evaluable)

Esta sección describe la visión de producto a futuro y no forma parte de los objetivos ni de la evaluación del trabajo. Se incluye únicamente para contextualizar el MVP como un subconjunto deliberado de un sistema mayor.

El MVP es la base de un producto más amplio de orquestación visual de agentes de código: un entorno donde el desarrollador planifica, ejecuta agentes en paralelo, anticipa conflictos antes del merge, inspecciona la evidencia técnica de cada agente (*diff*, logs, llamadas a herramientas, tests), ordena la integración con un *pull request* auditable y supervisa la salud del sistema (uso de recursos, costo). La hipótesis de negocio, a validar fuera de este trabajo, es que a medida que los agentes mejoran, el cuello de botella del desarrollo se desplaza de escribir código a coordinar trabajo de agentes, y que una herramienta que descompone, anticipa conflictos y mantiene control humano auditable tiene valor para equipos que operan repositorios reales. Los componentes diseñados en este trabajo son reutilizables como base de ese producto.

Apéndice B — Referencias

- *Guided Code Generation with LLMs* — [arXiv:2501.06625](https://arxiv.org/abs/2501.06625)
- *TDAG: Dynamic Task Decomposition and Agent Generation* — [arXiv:2402.10178](https://arxiv.org/abs/2402.10178)
- *Compositional Hardness of Code in LLMs* — [arXiv:2409.18028](https://arxiv.org/abs/2409.18028)
- *AgentOrchestra: Hierarchical Multi-Agent Intelligence* — [arXiv:2506.12508](https://arxiv.org/abs/2506.12508)
- *SWE-bench / SWE-bench Verified* — swebench.com
- *Symphony* (OpenAI, 2026) — openai.com/index/open-source-codex-orchestration-symphony/
- *The Code Agent Orchestra* (A. Osmani) — addyosmani.com/blog/code-agent-orchestra/
- *git-worktree* — git-scm.com/docs/git-worktree
- *TypeScript Compiler API* — github.com/microsoft/TypeScript/wiki/Using-the-Compiler-API